

PRACTICA 1:

Práctica 1: Implementación y Análisis de Bloques en GNU Radio para Acumulación y Diferenciación de Señales

León Rodríguez Silvia Fernanda; Silvia2215738@correo.uis.edu.co

Vilbao Parra Laura Fernanda; Laura2205123@correo.uis.edu.co

Abstract

This paper presents the implementation and validation of custom blocks in GNU Radio using Python. Accumulator and Differentiator blocks were developed to extract deterministic (amplitude, frequency, phase, period) and probabilistic (mean, variance, RMS) parameters from signals, allowing real-time analysis and improved noise handling. The results demonstrate the effectiveness of the approach and lay the foundation for future optimizations in software-defined radio systems.

1. INTRODUCCIÓN

Para diseñar un sistema de comunicaciones, es necesario comprender las características de la señal, conocidas como parámetros fundamentales: amplitud, frecuencia, fase y período, los cuales determinan el comportamiento del sistema [1].

A diferencia de las señales determinísticas, cuyo comportamiento se puede predecir con exactitud en cualquier instante de tiempo, las señales no tienen un patrón fijo y por eso no pueden anticiparse de la misma manera. Esta impredecibilidad hace necesario recurrir a parámetros probabilísticos. Entre ellos está la media, la varianza, el valor RMS, la potencia promedio y la desviación estándar. Además, como las señales cambian constantemente, su comportamiento no puede de una sola vez, sino que debe analizarse en segmentos cortos de tiempo. [2].

En la actualidad y con herramientas conocidas como **GNU Radio** se puede implementar mediante bloques programados en **Python**, lo que brinda mayor flexibilidad que los bloques prediseñados. Esto permite personalizar el análisis, optimizar el rendimiento del sistema, incorporar algoritmos avanzados y procesar señales en tiempo real. [1].

2. OBJETIVO PRINCIPAL

Implementar el uso de GitHub como herramienta de apoyo en el trabajo colaborativo, permitiendo el desarrollo, actualización y control de proyectos académicos de manera eficiente, mediante la creación de repositorios, el uso de tokens de acceso y la aplicación de buenas prácticas en el manejo de la información,

3. MARCO TEÓRICO

En la actualidad, el desarrollo de trabajos en entornos académicos y profesionales requiere de plataformas que permitan la organización y el control de proyectos de manera eficiente. GitHub es una de las herramientas más utilizadas para este fin, ya que se basa en **Git**, un sistema de control de versiones distribuido que facilita el seguimiento de cambios, la integración de aportes de diferentes usuarios y la gestión del código fuente.

El uso de GitHub no solo se limita al almacenamiento en la nube, sino que también permite implementar flujos de trabajo colaborativos mediante ramas, commits y pull requests, lo cual favorece la coordinación en proyectos de gran escala. Además, el manejo de **tokens de acceso personal** garantiza la seguridad en las operaciones realizadas desde repositorios locales hacia la plataforma en línea.

De igual manera, la correcta gestión de la información es un aspecto esencial en el desarrollo de proyectos académicos y de investigación. Contar con repositorios organizados y documentados (por ejemplo, a través de archivos *Readme.md*) asegura la trazabilidad y la reutilización del conocimiento generado, apoyando tanto el proceso de formación como el desarrollo de futuras investigaciones.

4. METODOLOGÍA EXPERIMENTAL

Inicialmente se crea una rama para el desarrollo de la práctica desde el repositorio, denominada *Práctica_1*, la cual debe ser compartida con los demás integrantes del grupo. A esta rama se le asigna un repositorio con el mismo nombre, y dentro de dicho repositorio se crean dos carpetas: GNU Radio e Informe.

Se implementaron dos bloques de Python en el entorno de GNU-Radio: un acumulador y un diferenciador, con ayuda del libro guía propuesto en la práctica, en la sección 1.2.0.1 [1], se obtuvieron los códigos guías para implementarlos en los respectivos bloques.

Bloque acumulador:

El bloque de Python denominado Acumulador se programó para sumar de manera consecutiva las muestras de la señal de entrada según la ecuación:

$$y[n] = y[n - 1] + x[n]$$

Para evitar que la salida permaneciera cercana a cero con una señal cuadrada simétrica, se aplicó un desplazamiento de -0.5 a la fuente. Este offset impide la cancelación entre valores positivos y negativos, lo que permite apreciar de manera clara la evolución acumulada. Finalmente, se conectó la salida del bloque y la señal de entrada a un visualizador en tiempo real, a fin de comparar la forma de onda original y la forma procesada.

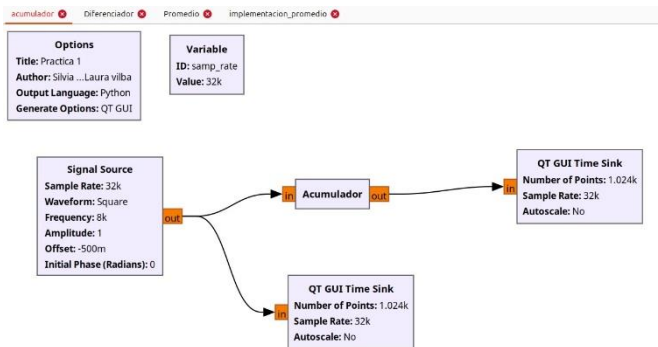


Figura 1. Implementación del bloque acumulador.

Bloque diferenciador:

El bloque diferenciador es aquel que realiza la operación de la diferenciación, es decir, calcula la derivada con respecto al tiempo, de manera matemática se puede ver de la siguiente manera.

$$y[n] = x[n] + x[n - 1]$$

Nuevamente se emplea un bloque de Python en GNU Radio para implementar un algoritmo que me haga esta operación, en donde se le ingresa una señal con el fin de obtener su derivada.

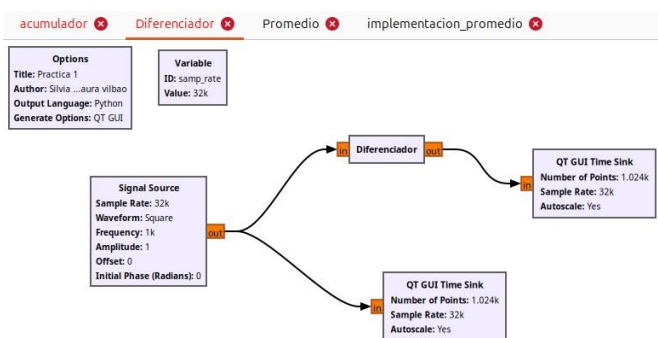


Figura 2. Implementación del bloque diferenciador.

Bloque promedio del tiempo

El bloque desarrollado, llamado “Promedios_de_tiempos”, fue definido como un bloque de sincronización (gr.sync_block). Se inicializó con variables acumulativas para mantener un seguimiento del cálculo en tiempo real, evitando pérdida de información en flujos continuos de datos.

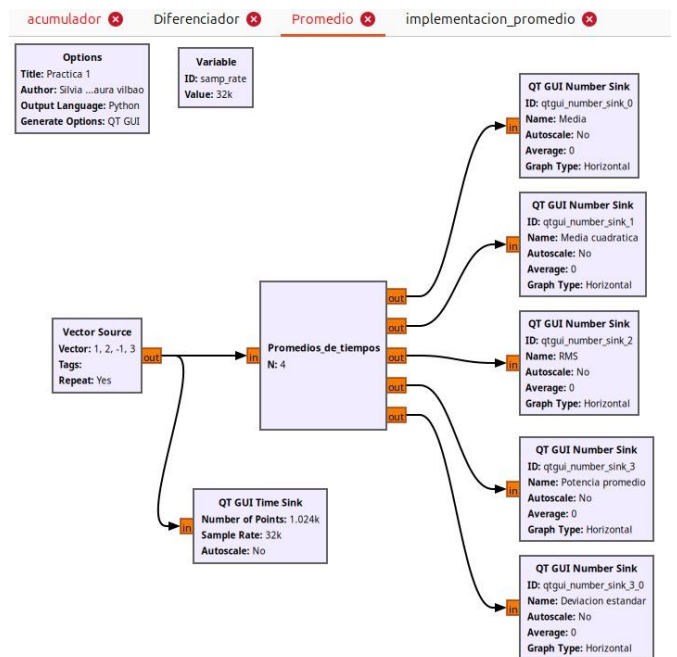


Figura 3. Implementación del bloque Promedio.

Dentro del método **work()** se implementaron los siguientes cálculos:

- **Media:** se obtiene sumando todos los valores de la señal y dividiéndolos entre el número total de muestras.
- **Media cuadrática:** se calcula de forma análoga a la media, pero considerando los valores de la señal al cuadrado.
- **Valor RMS:** corresponde a la raíz cuadrada de la media cuadrática.
- **Potencia promedio:** resulta de elevar al cuadrado el valor RMS.
- **Desviación estándar:** se determina a partir de la suma de las diferencias al cuadrado entre la señal y su media, dividiendo entre la cantidad de muestras y extrayendo finalmente la raíz cuadrada.

Con esta implementación se plantea una aplicación orientada a reducir la influencia del ruido gaussiano en los cálculos estadísticos. Para ello, la señal se segmenta en 10 ventanas de 4096 muestras cada una y se emplean funciones nativas de Python, con el fin de compararlas frente a las propuestas en el libro.

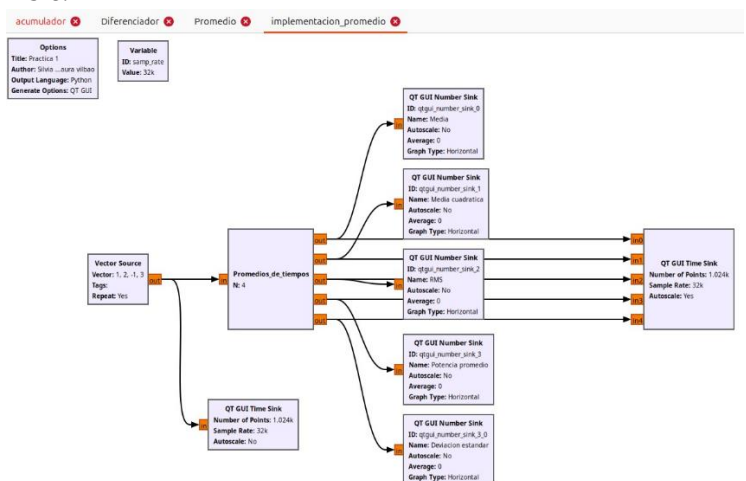


Figura 4. Implementación de la aplicación de reducción del ruido Gaussiano.

5. ANÁLISIS DE RESULTADOS

Bloque acumulador:

Para la prueba se utilizó una señal cuadrada con un desplazamiento de 0.5 como entrada. Al pasarla por el bloque **Acumulador**, la salida resultante tomó la forma de una onda triangular, debido a que cada nivel constante de la señal cuadrada se integra de manera lineal mientras permanece activo. Además, el desplazamiento aplicado evita la cancelación entre valores positivos y negativos, lo que provoca que la salida vaya aumentando progresivamente.

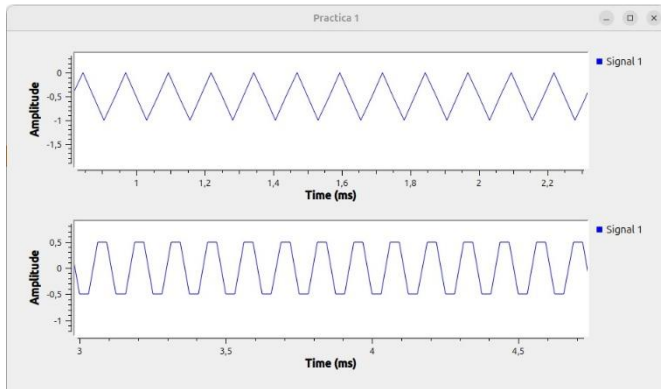


Figura 5. Resultados de simulación bloque Acumulador.

Bloque diferenciador:

Para comprobar el correcto funcionamiento del bloque diferenciador, se aplicó una señal cuadrada. Como era de esperar, la salida correspondió a una serie de impulsos, obteniéndose el siguiente resultado.

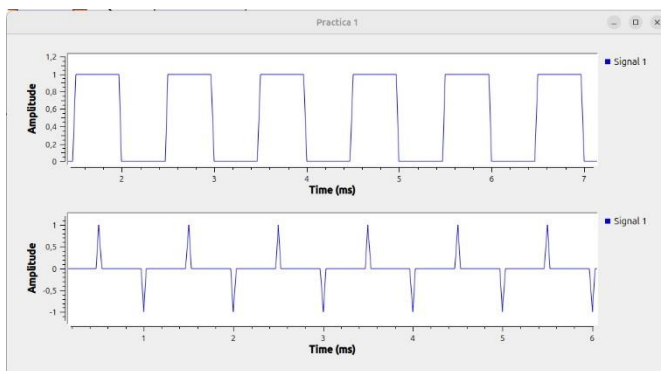


Figura 6. Resultados de simulación bloque Diferenciado.

Promedio de tiempo y aplicación:

El bloque **"Promedios_de_tiempos"** permitió comprobar el cálculo en tiempo real de las principales estadísticas de la señal. En la primera simulación, con una señal cuadrada de entrada, se observó que los valores obtenidos para la media, media cuadrática, RMS, potencia promedio y desviación estándar fueron coherentes con el comportamiento esperado de la señal periódica sin ruido.

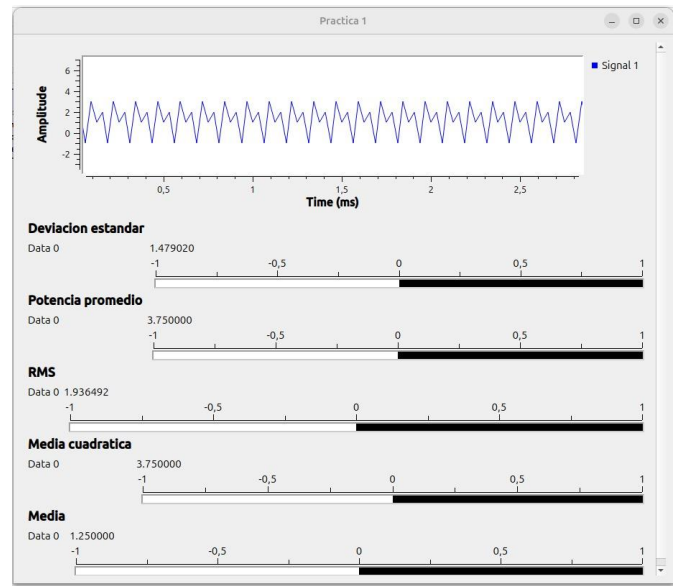


Figura 7. Resultados de simulación promedio de tiempo.

En esta prueba se utilizó una señal de entrada distinta a la anterior, de forma triangular/diente de sierra, con amplitud variando aproximadamente entre -1 y 3 . A diferencia de la Figura 7, en este caso se incluyó una segunda gráfica en la que se representan cinco señales constantes en el tiempo, correspondientes a cada uno de los parámetros estadísticos calculados por el bloque (media, media cuadrática, RMS, potencia promedio y desviación estándar), lo que permite visualizar que dichos valores se mantienen estables mientras la señal conserva su periodicidad.

Al comparar los resultados numéricos con los obtenidos en la Figura 7 (media de 1.25, media cuadrática y potencia promedio de 3.75, RMS de 1.936492 y desviación estándar de 1.479020), se observa que los valores no presentaron variación, a pesar de tratarse de una forma de onda diferente. Esto indica que, para ambas señales, el bloque "Promedios_de_tiempos" mantiene un cálculo consistente de las estadísticas, y que la representación en el tiempo (segunda gráfica) confirma la estabilidad de dichos parámetros a lo largo de la simulación.

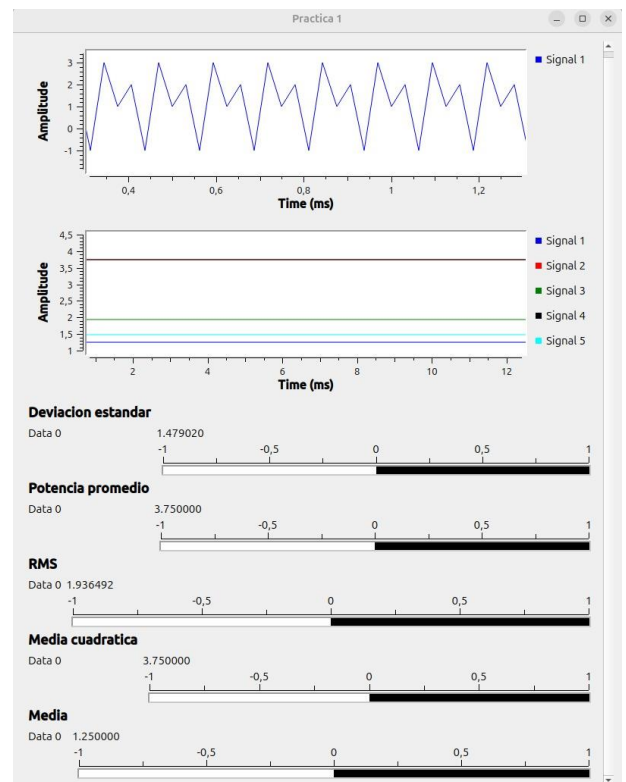


Figura 8. Resultados de simulación, aplicación.

6. CONCLUSIONES

- La configuración del bloque "Promedios_de_tiempos" como un bloque de sincronización (gr.sync_block) con variables acumulativas fue una elección de diseño acertada para evitar la pérdida de datos en flujos continuos. El proceso comprobó que los algoritmos programados mantienen un cálculo estable y consistente de los parámetros probabilísticos (media, RMS, desviación estándar y potencia), independientemente de las variaciones en la forma de onda de entrada (cuadrada o triangular).
- Durante el proceso de acumulación, la aplicación estratégica de un desplazamiento (offset) de -0.5 a la señal cuadrada fue un paso crítico para el éxito de la práctica. Esta configuración impidió la cancelación de los valores positivos y negativos, permitiendo integrar la señal de forma lineal y evidenciar correctamente su transformación en una onda triangular.
- El enfoque metodológico de segmentar la señal en 10 ventanas de 4096 muestras resultó ser una técnica de procesamiento viable para reducir la influencia del ruido gaussiano en los cálculos estadísticos. Este procedimiento sienta las bases para futuras optimizaciones y un manejo más robusto de señales impredecibles en sistemas de radio definida por software,
- La programación de bloques propios en Python dentro de GNU Radio (Acumulador y Diferenciador) demostró otorgar una mayor flexibilidad y personalización frente al uso de bloques prediseñados. Este proceso de desarrollo propio permitió ajustar matemáticamente el comportamiento del sistema para analizar las señales en tiempo real.

7. REFERENCIAS

- [1] H. O. Boada and O. M. R. Torres, Comunicaciones Digitales basadas en radio definida por software, primera ed. Bucaramanga, Colombia: Editorial UIS, 2019.
- [2] S.-M. M. Yang, Review of Signals and Systems, Signals and systems, and of Probability and Random Process. Cham: Springer International Publishing, 2020. [Online]. Available: https://doi.org/10.1007/978-3-030-57706-3_9

